

Rajalakshmi Engineering College

Name: Sheik Abdulla
Email: 240701493@rajalakshmi.edu.in
Roll no: 240701493
Phone: 6379676782
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 3_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Emma is a data analyst working with a grid-based system where each cell contains important numerical data. The grid represents spatial data, inventory records, or structured reports that require periodic updates.

Due to system updates and new requirements, Emma needs to modify the grid in the following ways:

She wants to insert either a new row or a new column at a given position. Later, she needs to delete either a row or a column from the modified matrix.

Input Format

The first line contains two integers rows and cols (the dimensions of the matrix).

The next rows lines contain cols space-separated integers representing the initial matrix.

The next line contains two integers insertType and insertIndex:

- insertType = 0 for row insertion, 1 for column insertion.
- insertIndex is the position where the new row/column should be added.

If inserting a row, the next cols integers represent the new row or If inserting a column, the next rows integers represent the new column.

The next line contains two integers deleteType and deleteIndex:

- deleteType = 0 for row deletion, 1 for column deletion.
- deleteIndex is the position to be deleted.

Output Format

The first line of output prints the string "After insertion" followed by the modified matrix with the inserted row or column.

Each row of the matrix is printed on a new line with space-separated integers.

The next line prints the string "After deletion" followed by the final matrix after the specified deletion operation.

Each row of the resulting matrix is printed on a new line with space-separated integers.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 3

1 2 3

4 5 6

7 8 9

0 1

10 11 12

12

Output: After insertion

1 2 3
10 11 12
4 5 6
7 8 9
After deletion
1 2
10 11
4 5
7 8

Answer

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        int rows = sc.nextInt();  
        int cols = sc.nextInt();  
  
        int[][] matrix = new int[rows][cols];  
  
        // Read initial matrix  
        for (int i = 0; i < rows; i++)  
            for (int j = 0; j < cols; j++)  
                matrix[i][j] = sc.nextInt();  
  
        int insertType = sc.nextInt(); // 0 for row, 1 for column  
        int insertIndex = sc.nextInt();  
  
        if (insertType == 0) {  
            // Insert row  
            int[] newRow = new int[cols];  
            for (int i = 0; i < cols; i++) {  
                newRow[i] = sc.nextInt();  
            }  
  
            // New matrix size  
            int[][] temp = new int[rows + 1][cols];  
  
            int r = 0;  
            for (int i = 0; i < rows + 1; i++) {
```

```

        if (i == insertIndex) {
            // Insert new row here
            for (int c = 0; c < cols; c++) {
                temp[i][c] = newRow[c];
            }
        } else {
            // Copy old rows
            for (int c = 0; c < cols; c++) {
                temp[i][c] = matrix[r][c];
            }
            r++;
        }
    }
    matrix = temp;
    rows++;
} else {
    // Insert column
    int[] newCol = new int[rows];
    for (int i = 0; i < rows; i++) {
        newCol[i] = sc.nextInt();
    }

    // New matrix size
    int[][] temp = new int[rows][cols + 1];

    for (int r = 0; r < rows; r++) {
        int c2 = 0;
        for (int c = 0; c < cols + 1; c++) {
            if (c == insertIndex) {
                temp[r][c] = newCol[r];
            } else {
                temp[r][c] = matrix[r][c2];
                c2++;
            }
        }
    }
    matrix = temp;
    cols++;
}

// Print matrix after insertion
System.out.print("After insertion ");

```

```

printMatrix(matrix);

int deleteType = sc.nextInt(); // 0 for row, 1 for column
int deleteIndex = sc.nextInt();

if (deleteType == 0) {
    // Delete row
    int[][] temp = new int[rows - 1][cols];
    int r2 = 0;
    for (int i = 0; i < rows; i++) {
        if (i == deleteIndex) continue;
        for (int c = 0; c < cols; c++) {
            temp[r2][c] = matrix[i][c];
        }
        r2++;
    }
    matrix = temp;
    rows--;
} else {
    // Delete column
    int[][] temp = new int[rows][cols - 1];
    for (int r = 0; r < rows; r++) {
        int c2 = 0;
        for (int c = 0; c < cols; c++) {
            if (c == deleteIndex) continue;
            temp[r][c2] = matrix[r][c];
            c2++;
        }
    }
    matrix = temp;
    cols--;
}

// Print matrix after deletion
System.out.print("After deletion ");
printMatrix(matrix);

sc.close();
}

private static void printMatrix(int[][] matrix) {
    for (int[] row : matrix) {

```

```
        for (int val : row) {  
            System.out.print(val + " ");  
        }  
        System.out.print("\n");  
    }  
    System.out.println();  
}  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement:

Emma, a budding computer vision enthusiast, is working on a challenging image processing project. She has a square image represented as a 2D matrix of integers. As part of a special filter operation, she needs to rotate the image by 90 degrees clockwise, but there's a twist — she must perform the rotation in-place, using no extra space.

This means Emma has to rotate the matrix without creating a new one. Your task is to help her implement a Java program that takes this square matrix as input and rotates it within the same structure.

Can you help Emma efficiently rotate the image so that her project can move to the next stage?

Input Format

The first line of input contains a single integer n , representing the number of rows and columns of the square matrix (i.e., the matrix is of size $n \times n$).

The next n lines each contain n space-separated integers, representing the elements of each row of the 2D array.

Output Format

The first line of output prints "Rotated 2D Array:"

The next n lines of output print the rotated matrix.

Each line contains n space-separated integers representing a row of the rotated matrix.

Refer to the sample output for format specification.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: Rotated 2D Array:

7 4 1

8 5 2

9 6 3

Answer

```
import java.util.Scanner;
```

```
public class Main{  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        // Read matrix size  
        int n = sc.nextInt();  
        int[][] mat = new int[n][n];  
  
        // Read matrix elements  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                mat[i][j] = sc.nextInt();  
            }  
        }  
    }
```

```
        // Step 1: Transpose the matrix  
        for (int i = 0; i < n; i++) {  
            for (int j = i + 1; j < n; j++) {  
                int temp = mat[i][j];  
                mat[i][j] = mat[j][i];  
                mat[j][i] = temp;  
            }  
        }  
    }
```

```

// Step 2: Reverse each row
for (int i = 0; i < n; i++) {
    int left = 0, right = n - 1;
    while (left < right) {
        int temp = mat[i][left];
        mat[i][left] = mat[i][right];
        mat[i][right] = temp;
        left++;
        right--;
    }
}

// Output
System.out.println("Rotated 2D Array:");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(mat[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Nikila is working as an intern in a software firm and is practicing with a matrix where each row represents a set of numerical values. Her task is to identify the row with the highest sum of its elements and remove that row from the matrix. After removing the row with the highest sum, Nikila needs to print the updated matrix.

Your task is to help Nikila in implementing the same. If there are two or more rows that have same the highest sum, the firstly encountered row is deleted.

Input Format

The first line of the input consists of two space-separated integers, R and C, representing the number of rows and columns in the matrix, respectively.

The following R lines each contain, C space-separated integers representing the matrix elements.

Output Format

The output prints the matrix after removing the row with the highest sum. Each row should be printed on a new line, with elements separated by a space.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 2 2

1 2

3 4

Output: 1 2

Answer

```
import java.util.Scanner;
```

```
public class Main{  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
        // Read R and C  
        int R = sc.nextInt();  
        int C = sc.nextInt();  
        int[][] matrix = new int[R][C];
```

```
        // Read the matrix  
        for (int i = 0; i < R; i++) {  
            for (int j = 0; j < C; j++) {  
                matrix[i][j] = sc.nextInt();  
            }  
        }  
    }  
}
```

```

// Find the row with the maximum sum
int maxSum = Integer.MIN_VALUE;
int maxRowIndex = 0;

for (int i = 0; i < R; i++) {
    int rowSum = 0;
    for (int j = 0; j < C; j++) {
        rowSum += matrix[i][j];
    }
    if (rowSum > maxSum) {
        maxSum = rowSum;
        maxRowIndex = i;
    }
}

// Print the matrix without the max sum row
for (int i = 0; i < R; i++) {
    if (i == maxRowIndex) continue; // skip the row to be removed
    for (int j = 0; j < C; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}

sc.close();
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Rina is managing the inventory for a library, where each row of a 2D matrix represents the number of different genres of books available on each shelf.

She wants to perform the following operations:

Transformation: Replace each element in a row with the sum of all elements in that row. **Merging:** After transformation, Rina will provide one

additional matrix, and specify whether to merge the transformed matrix with this new matrix row-wise or column-wise.

Input Format

The first line contains two integers R and C, representing the number of rows and columns of the initial matrix.

The next R lines contain C space-separated integers, representing the book counts in the library.

The next line contains two integers MR and MC, representing the dimensions of the second matrix (to be merged).

The next MR lines contain MC space-separated integers, representing the second matrix.

The last line contains an integer mergeType:

- 0 Row-wise merging (append the second matrix below the transformed matrix).
- 1 Column-wise merging (append the second matrix to the right of the transformed matrix).

Output Format

The output prints "Transformed matrix: " followed by the transformed 2D matrix where each element in a row is replaced with the sum of the elements in that row.

The output prints "Final merged matrix: ", followed by the merging based on mergeType.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3 4
8 2 4 9
4 5 6 1
7 8 9 3

2 4
3 5 7 2
6 1 4 9
0

Output: Transformed matrix:

23 23 23 23
16 16 16 16
27 27 27 27

Final merged matrix:

23 23 23 23
16 16 16 16
27 27 27 27
3 5 7 2
6 1 4 9

Answer

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Read dimensions and initial matrix
        int R = sc.nextInt();
        int C = sc.nextInt();
        int[][] mat1 = new int[R][C];
        for (int i = 0; i < R; i++)
            for (int j = 0; j < C; j++)
                mat1[i][j] = sc.nextInt();

        // Transform the matrix
        for (int i = 0; i < R; i++) {
            int rowSum = 0;
            for (int j = 0; j < C; j++) {
                rowSum += mat1[i][j];
            }
            for (int j = 0; j < C; j++) {
                mat1[i][j] = rowSum;
            }
        }
    }
}
```

```

// Read second matrix
int MR = sc.nextInt();
int MC = sc.nextInt();
int[][] mat2 = new int[MR][MC];

for (int i = 0; i < MR; i++)
    for (int j = 0; j < MC; j++)
        mat2[i][j] = sc.nextInt();

int mergeType = sc.nextInt();

// Print transformed matrix
System.out.print("Transformed matrix: ");
for (int i = 0; i < R; i++)
    for (int j = 0; j < C; j++)
        System.out.println(mat1[i][j] + " ");
System.out.println();

// Merge matrices
System.out.print("Final merged matrix: ");
if (mergeType == 0) { // Row-wise
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            System.out.println(mat1[i][j] + " ");
    for (int i = 0; i < MR; i++)
        for (int j = 0; j < MC; j++)
            System.out.println(mat2[i][j] + " ");
} else if (mergeType == 1) { // Column-wise
    int maxRows = Math.max(R, MR);
    for (int i = 0; i < maxRows; i++) {
        // Print from transformed matrix
        if (i < R) {
            for (int j = 0; j < C; j++)
                System.out.println(mat1[i][j] + " ");
        }

        // Print from second matrix
        if (i < MR) {
            for (int j = 0; j < MC; j++)
                System.out.println(mat2[i][j] + " ");
        }
    }
}

```

```
}  
    sc.close();  
}  
}
```

Status : Correct

Marks : 10/10